AI Research Engineer — Interview Preparation Course

A course built for one purpose: passing the technical and coding interviews at an AI research lab. Eleven self-contained modules take you from the mathematics the questions rest on, through PyTorch as the working language, through how models are trained, scaled, served and aligned, down to CUDA at the level below the framework.

Each module is an article that explains the topic, a further reading list that says which paper to open first and why, and a programming exercise that grades itself.

Every claim in every article is either derived or measured. Every exercise checks itself.

The modules

#	Module	What you build
00	PyTorch as a Numerical Language	Strides, broadcasting, autograd as VJPs, a stable softmax, multi-head attention
01	Theoretical Foundations	Eckart–Young, conditioning vs. convergence rate, softmax from max-entropy, attention's gradient by hand
02	Implementation From Scratch	A transformer in numpy, forward *and* backward, that learns a task; k-means++, O(n log n) AUC
03	ML Debugging	Eight planted bugs in a working model, and the diagnostics that find them
04	ML System Design	Memory budgets, tensor/data/pipeline parallelism verified in simulation, the GPipe bubble derived
05	Inference Optimisation	KV cache, quantisation at three granularities, speculative decoding proven distribution-preserving
06	RAG Systems	BM25, dense retrieval, nDCG, RRF fusion, an IVF index, reranking, citation verification
07	Research & Paper Analysis	FlashAttention reproduced from its recurrence, plus a structured critique
08	AI Safety & Alignment	The policy gradient derived and measured, PPO's clip read case by case, reward hacking reproduced, DPO derived, the fairness impossibility theorem demonstrated
09	CUDA and cuBLAS	The SGEMM ladder against cuBLAS, warp reductions, a fused attention kernel
10	Mixture of Experts	Mixtral's parameter count reproduced, routing collapse and its cure, expert parallelism, the decoding penalty

GLOSSARY.md collects the notation used across the modules: the kinds of multiplication, the marks on symbols, the norms, the Greek letters and the abbreviations.

The spine: one function, eleven levels

One task, revisited at every level, runs through the whole course. **Scaled dot-product attention** appears in every module, each time from a different angle. By the end you will have seen the same twenty lines of

mathematics as tensor code, as a derivation, as hand-written gradients, as a bug, as a distributed computation, as a latency problem, as a retrieval system, as a research paper, as an alignment target, as a CUDA kernel, and finally as the one part of a block that a mixture of experts leaves alone.

module	attention as…	the connection back
00	tensor operations	implement it with broadcasting and a stable softmax
01	a low-rank bilinear form	$W_Q W_K^\top$ has rank $\le d_{\text{head}}$; derive $\bar{Q}, \bar{K}, \bar{V}$
02	code you own	numpy, forward and backward, no autograd underneath
03	four of the eight planted bugs	the causality test from 00 is what catches them
04	a sharded computation	heads are already independent, so the partition follows the maths
05	a cache and a quantisation target	the causal mask from 03 is *why* a KV cache is exact
06	soft retrieval	softmax over a context vs. top-k over an index
07	a paper	FlashAttention's backward rests on the rowsum lemma you proved in 01
08	the thing being aligned	a judge replaces the missing loss, reinforcement learning turns its score into a gradient, and every failure follows from that
09	a kernel	the tiling from 07, with the memory hierarchy made explicit
10	the part that stays dense	the MLP goes sparse, attention does not, and the all-to-all sits between them

Two threads are worth following in particular, because they close loops across four modules each:

- **The rowsum identity** $\text{rowsum}(\bar{P} \odot P) = \text{rowsum}(\bar{O} \odot O)$ is a throwaway sub-step in exercise 01, and turns out to be the lemma that lets FlashAttention's backward pass run in linear memory (07).
- **Causality** is asserted as a test in module 03, is the reason the KV cache is an exact identity rather than an approximation in module 05, and determines the tile masking in the CUDA kernel in module 09.

Setup

```
python -m venv .venv
.venv/Scripts/activate            # Windows;  source .venv/bin/activate on Linux/macOS
pip install -r requirements.txt
```

Everything runs on CPU. A GPU helps modules 05 and 09 and is required for the CUDA half of 09; module 09's notebook has a substantial GPU-free half that does not.

No model downloads are needed anywhere. Module 06 builds its dense retriever from a TF-IDF matrix reduced by SVD, and retrieves over this repository's own articles.

How each module is laid out

```
NN-module-name/
  README.md           the article  (~5 pages; ends with a references list)
```

```
NN-module-name.pdf  the article, typeset — generated from README.md
exercise.py         the exercise, in percent-cell format (the source of truth)
exercise.ipynb      the notebook, generated from exercise.py
solution.py         reference solutions; `python solution.py` runs every check
```

The root `README.md` likewise has a `README.pdf`. PDFs are rebuilt with `python tools/build_pdfs.py` (pandoc + XeLaTeX; `--force` to rebuild everything).

Module 03 also ships `buggy_lm.py` (the model you repair); module 06 ships `corpus.py`; module 09 ships `kernels/` with the CUDA sources and build scripts.

Notebooks are generated, not edited. Edit `exercise.py`, then:

```
python tools/build_notebooks.py              # rebuild all
python tools/build_notebooks.py 05-inference-optimization
python tools/build_notebooks.py --check      # fail if any notebook is stale
```

Plain `.py` is what lives in the repository: it diffs cleanly, runs without a kernel, and cannot accumulate stale outputs.

How to use this

Read the article first. The exercises assume it; several checks are stated in the article's vocabulary and will look arbitrary without it.

Attempt before reading `solution.py`. Every exercise has `TODO` stubs and a check harness that prints pass/fail without aborting, so you can work incrementally. The solutions are commented with *why* a particular choice was made, and what the alternative was — that is the part worth reading after you have your own version working.

Verify as you go. Run `python solution.py` in any module to see the full set of checks pass:

```
$ python 01-math-foundations/solution.py
[ok]   3 κ=10000  -- 0.99980002 vs 0.99980002
[ok]   6b saturation shrinks the gradient  -- 1.553e+00 -> 4.300e-01
...
All 31 checks passed.
```

Time-box. Interviews are time-boxed; practising without a clock trains the wrong thing. Rough budgets: article 30 minutes, exercise 2–4 hours, discussion questions 30 minutes.

A suggested order

If you are preparing for a specific loop rather than working through linearly:

- **Coding round soon:** 00 → 02 → 03.
- **Systems round soon:** 04 → 05 → 09, and 10 if the role touches sparse models.
- **Research round soon:** 01 → 07, and re-read your own papers with §4 of module 07 in hand.
- **Anything at Anthropic/OpenAI/DeepMind:** 08 is not optional, and it is the one most people underprepare.

Exercise style

The checks are the pedagogy. They are written to fail for a *reason* rather than on a tolerance:

- **Exactness where exactness is the claim.** The KV cache, tiled attention and tensor-parallel sharding are algebraic identities, and are checked at `atol=1e-13`, not `1e-3`. If your version is merely close, you have a bug.
- **Invariants over values.** Causality, attention weights summing to one, "every output element written exactly once" — these catch bugs that eyeballing never will.
- **Honest negatives.** Several checks assert that a *wrong* approach fails: that a missing all-reduce is detectable, that one outlier wrecks per-tensor quantisation, that RRF is not a rank average. Knowing the failure mode is the transferable part.
- **Nuance where the textbook answer is too strong.** Module 07 shows FlashAttention's traffic advantage converges to a constant rather than growing with sequence length; module 08 shows the robustness/accuracy trade-off is conditional on whether the fragile features are predictive. Being able to state those precisely is what the interview is actually testing.

Status

All ten articles and exercises are complete, and every `solution.py` passes on CPU:

module	checks
00 pytorch-basics	26
01 math-foundations	31
02 ml-coding-from-scratch	22
03 ml-debugging	9 (2/9 on the un-fixed model, by design)
04 ml-system-design	44
05 inference-optimization	26
06 rag-systems	29
07 research-paper-analysis	30
08 ai-safety-alignment	30
09 cuda-and-cublas	25 (the GPU-free half)
10 mixture-of-experts	26

The CUDA sources in `09-cuda-and-cublas/kernels/` have **not** been compiled or run — this machine has the CUDA toolkit but no host C++ compiler. Their index arithmetic *is* verified: module 09's notebook emulates the exact indexing of the tiled GEMM, the register-tiled GEMM and the fused attention kernel in numpy and checks them against reference results. Treat the timing tables as untested and the algorithms as verified.